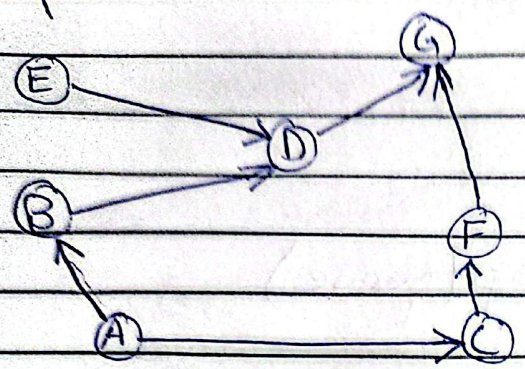
Raad Khan FA24-BCS-009

Graph:



(Q1):

1) Start: Priority Queue = [A(0)]
2) Expand A: Neighbours B(1), C(4), F(5) → Queue = [B(1), C(4), F(5)]
3) Expand B(1): Neighbors D(2), E(4) → Queue = [D(2), C(4), E(4), F(5)]
4) Expand D(2): Neighbors E(3), G(4) → Queue = [E(3), C(4), G(4), E(4), F(5)]
5) Expand E(3): No new nodes → Queue = [C(4), G(4), E(4), F(5)]
6) Expand G(4): Goal Found

(Q2):

```
import heapq
class Graph:
  def_init_(self):
  self.graph = {
        'A' : [('B',1), ('C',4), ('F',5)],
        'B' : [('D',1), ('E',3)],
        'C' : [],
        'D' : [('E',1), ('G',2)],
        'E' : [],
        'F' : [],
        'G' : [] }
```

```
def ucs(self, start, goal):
    queue = [(0, start, [start])]
    visited = set()

while queue:
    cost, node, path = heapq.heappop(queue)
    if node in visited:
        continue
    visited.add(nod)
    if node == goal:
        return path, cost
for neighbor, edge_cost in self.graph[node]:
    if neighbor not in visited:
```

(Q3):

Key insight: For this graph all UCS, BFS, DFS all return the same path but UCS is the correct algorithm becaue edge costs differ.

(Q4):

1) What is start node? = A
2) What is goal node? = G
3) What is UCS path from A→G? = A-B-D-G
4) What is UCS cost? = 4
5) What is BFS Path? = A-B-D-G
6) Does BFS guarantee least cost? = Yes
7) Which node is expanded first in UCS? = A
8) What is cost from A→D? = 2
9) Is there path for A→G through C? = No
10) How many nodes are expanded before G? = 3
11) Is graph Directed? Yes
12) Can DFS give a different path?
13) What is the cost of A→B→E?
14) Which algorith is Best for weighted graphs.
15) What is heuristic value needed for A*.